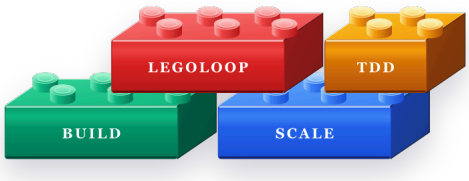
MANUAL OPERACIONAL DIDÁTICO · VIDEO AS CODE

video-001

Edição por Roteiro 100% Local

Edição por roteiro com Whisper, HyperFrames e FFmpeg, sem APIs pagas



SOBRE ESTA OBRA

Documento gerado deterministicamente a partir das fontes citadas no próprio texto, via pandoc + typst. Toda afirmação técnica desta obra é rastreável a um arquivo nomeado no capítulo correspondente.

Data da compilação: 08/11/2026 · LegoLoop Foundation

Introdução

Este livro documenta o pipeline `video-001`, derivado do vídeo "Automatizei a edição dos meus vídeos com Claude Code (passo a passo)", de Chiara Costa. O pipeline recebe um vídeo bruto e um roteiro, e entrega um vídeo final em 1080p60 com animações em código, sem chamar serviços pagos.

1. Fundação e Princípios

O problema

Editar vídeos gravados e criar animações é lento. O fluxo original usa Claude Code com a skill HyperFrames, narração com voz clonada e cenas realistas geradas por serviço externo. ## O que este pipeline decide | Decisão | Escolha | | :———————— | : ———————————————————— | | Transcrição | `whisper-cli` local com modelo `ggml-small` | | Animações | HyperFrames local, HTML renderizado para WebM com alfa | | Montagem | FFmpeg, saída 1920x1080 a 60 fps | | Voz | Áudio do operador; sem clonagem por API | | Revisão humana | Plano de cenas aprovado antes de qualquer render | ## Fora do escopo Serviços de nuvem pagos, voz neural por API externa, cenas realistas de gerador de vídeo e interface web própria.

2. Matriz Original e Curadoria

Stack declarada no vídeo

Ferramenta	Papel	Custo declarado	Substituição local
Claude Code com Opus 5·5	Orquestra a edição	R$ 100,00 por mês (plano)	Gateway local 9Router
Higgsfield Plus	Cenas realistas	Não informado no vídeo	Opcional, fora do núcleo
HeyGen	Voz clonada	Não informado no vídeo	Áudio do operador
ElevenLabs	Voz por API (alternativa)	Não informado no vídeo	Áudio do operador
HyperFrames	Animações em código	Gratuito	Mantido
Remotion	Alternativa ao HyperFrames	Licença comercial conforme porte	Dispensado
FFmpeg, Node.js, Python	Base técnica	Gratuito	Mantidos

Consumo observado

Uma edição consumiu cerca de 10% do limite do plano (de 35% para 45%). O vídeo citado somou mais de 76 mil visualizações orgânicas. ## Diagnóstico do host `ffmpeg`, `node`, `python` e `whisper-cli` já estavam instalados. O `hyperframes` estava com a instalação quebrada (link para um clone apagado) e foi reinstalado com `bun install -g hyperframes`.

3. Método Autoral

O método segue cinco regras observadas no vídeo.

1. **Contexto antes do pedido.** Quanto mais contexto o prompt carrega (pasta do produto, roteiro, referência), melhor o resultado. Pedidos soltos falham.
2. **Estrutura do prompt.** O pedido vem no início e o contexto necessário vem logo abaixo.
3. **Referência de estilo.** Um vídeo de referência indica como a edição deve ficar.
4. **Planejar antes de gerar.** O modelo escreve o plano cena por cena e o operador corrige antes da geração.
5. **Transformar correções em regras.** O estilo e os ajustes pedidos viram uma skill reutilizável.

Prompt-base do vídeo

> *Crie um vídeo de apresentação desse produto. Use motion graphics e mostre todo o seu potencial para produção de vídeo.*

4. Arquitetura do DAG

O DAG tem oito nós em cadeia linear. | Nó | Tipo | Função | Saída | | :———————— | :———— | :———————————— | :———————— | | `n01_extrair_audio` | script | FFmpeg extrai áudio 16 kHz mono | `audio.wav` | | `n02_transcrever` | script | `whisper-cli` com `-ojf` | `audio.json` | | `n03_alinhar_roteiro` | script | Casa as falas do roteiro às palavras | `segmentos.json` | | `n04_plano_cenas` | skill | Escreve o plano cena por cena | `plano.md` | | `n05_gate_plano` | stop_gate | Aprovação humana do plano | `plano_aprovado.json` | | `n06_renderizar_cenas` | script | HyperFrames renderiza WebM com alfa a 60 fps | `cenas.webm` | | `n07_montar` | script | FFmpeg sobrepõe as cenas ao vídeo base | `final.mp4` | | `n08_verificar` | script | `verificar_video.py` confere o resultado | `verificacao.json` | ## Contrato de `segmentos.json` Lista de objetos com `id`, `texto`, `inicio_ms` e `fim_ms`. O fim é sempre maior que o início e não há sobreposição entre segmentos. ## Alinhamento A função `alinhar` ignora acentos, caixa e pontuação, procura as duas primeiras palavras de cada fala numa janela de 80 palavras e falha com erro se a fala não existir na transcrição.

5. Operação e Limites

Executar

`legoloop run loops/video-001/pipeline.dag.json --var raw_video="caminho/do/video.mp4"` ## Pré-requisitos `ffmpeg` com `libvpx-vp9`, `node`, `python`, `hyperframes` 0.9.x com telemetria desativada (`hyperframes telemetry disable`) e `whisper-cli` com `ggml-small.bin`. ## Verificação do Golden Fixture O fixture `fixture_3s.mp4` tem 1920x1080, 60 fps, 3,0 s e áudio. O comando `python -I scripts/verificar_video.py --video <arquivo> --duracao 3` retorna exit 0 quando conforme. ## Limites conhecidos | Limite | Efeito | | : ——————————————— | :———————————— | | `n08` com duração fixa de 3 s | Ajustar para vídeos reais | | `n06` gera um único WebM | Não há um arquivo por cena | | Caminhos do Whisper fixos no `n02` | Específicos deste host | | DAG não executado de ponta a ponta pelo `legoloop run` | Fixture montado nó a nó | | Validação do `whisper-cli` | Apenas sobre 40 s de áudio real |